

CoreOps Week 6 Modules

OBJECTIVE: Learn to ship secure workloads with CI/CD pipelines, implement basic DevSecOps guardrails, understand GitOps workflows, and explore internal developer platform fundamentals.

MONDAY: GitHub Actions / GitLab CI Pipelines with Terraform & Docker

Critical Concepts

- CI pipeline stages (build → test → deploy)
- Docker build basics
- Terraform workflow (init/plan / apply)
- Storing secrets securely (GitHub Secrets / GitLab Variables)
- Basic pipeline caching

Key Outcomes

- You can write a simple CI pipeline for Docker + Terraform
- You can securely pass secrets into pipelines
- You understand how CI validates infrastructure changes

WHAT TO PRACTICE

THE DEVOPS WAR ROOM

Lab 1: Create a GitHub Action / GitLab CI pipeline

Steps:

1. Build a Docker image
2. Test the container (curl, simple script)
3. Push image to registry

Lab 2: Add Terraform workflow

- Validate Terraform format
- Run terraform plan
- Post plan results as pipeline output

Lab 3: Use pipeline caching

- Cache ~/.terraform
- Cache Docker layers

WHERE TO STUDY

- GitHub Actions docs
- GitLab CI yaml guide
- HashiCorp Terraform workflow

INFRATHrone MINDSET NOTES

- CI is not for deployment; CI is for **confidence**.
- Pipelines should fail fast; early errors save hours later.
- Automation works only when humans stop bypassing it.



TUESDAY: Image Signing + SBOM + Vulnerability Scanning

Critical Concepts

- What is an SBOM (Software Bill of Materials)?
- Container vulnerability scanning (Trivy, Grype)
- Cosign image signing
- Why image provenance matters
- Why DevSecOps ≠ blocking the pipeline everywhere

Key Outcomes

- You can generate an SBOM
- You can scan an image for vulnerabilities
- You can sign and verify container images
- You understand how to shift security left

WHAT TO PRACTICE

Lab 1: Generate SBOM using Syft or Docker Scout

- Export SBOM in JSON
- Track dependencies inside container

Lab 2: Scan vulnerabilities using Trivy

- Identify CVEs
- Set severity threshold (block only CRITICAL) ROOM

Lab 3: Sign image using Cosign

- Push image → sign → verify signature

WHERE TO STUDY

- sigstore.dev (Cosign docs)
- OWASP container security cheat sheet
- Trivy documentation

INFRAThrone MINDSET NOTES

- Security is a **quality dimension**, not a department.
- SBOM helps you understand **what you really shipped**.
- Sign everything, trust nothing by default.

WEDNESDAY: ArgoCD GitOps, Sync Strategies & Drift Detection

Critical Concepts

- GitOps model: Git = source of truth
- ArgoCD app structure (App CRD, repo URL, path)
- Pull-based deployment
- Auto-sync vs manual sync
- Drift detection (Git vs cluster state mismatch)

Key Outcomes

- You can create an ArgoCD Application
- You understand how GitOps sync works
- You can diagnose drift and reconcile application state

WHAT TO PRACTICE

Lab 1: Deploy an app with ArgoCD

- Create a Git repo with Kubernetes manifests
- Add ArgoCD Application pointing to repo
- Sync app

Lab 2: Induce drift

Manually edit something inside the cluster: `kubectl edit deploy myapp`
Then observe the ArgoCD drift alert.

Lab 3: Auto-sync and self-healing

Enable auto-sync → see cluster correct itself automatically.

WHERE TO STUDY

- ArgoCD official documentation
- CNCF GitOps principles

INFRAThrone MINDSET NOTES

- “kubectl apply” is not GitOps.
- Drift is inevitable; the goal is automatic correction.
- GitOps reduces outages caused by “who changed this?”

THURSDAY: Platform as a Product: Internal Developer Platforms

Critical Concepts

- What is a platform team?
- Golden paths (predefined delivery patterns)
- Self-service provisioning
- Abstractions vs complexity hiding
- Developer experience (DX) as a product metric

Key Outcomes

- You understand platform design philosophy
- You know why IDPs reduce operational load
- You see how templates accelerate secure delivery

WHAT TO PRACTICE

Lab 1: Design a Golden Path Template

Create a template defining:

- app scaffold
- Dockerfile
- CI workflow
- deployment manifest

Lab 2: Create a self-service workflow

A simple script or template that:

- provisions namespace
- applies resource quotas
- applies RBAC

Lab 3: Evaluate developer friction points

Document:

- where onboarding slows
- where manual work exists
- where automation can help

WHERE TO STUDY

- Team Topologies (Platform team patterns)
- Spotify Backstage blogs
- Humanitec IDP guides

INFRATHrone MINDSET NOTES

- Platform teams build **products**, not infra.
- A platform is successful only when devs prefer it over DIY.
- Golden paths prevent 80% of security misconfigurations.



FRIDAY: Backstage + Scorecards + Self-Service Tools

Critical Concepts

- Backstage software catalog
- TechDocs
- Scorecards (best-practice enforcement)
- Self-service actions (scaffolder)
- Plugin-based architecture

Key Outcomes

- You can register a service in Backstage
- You can create a simple template
- You can generate scorecards for security & quality
- You understand self-service provisioning patterns

WHAT TO PRACTICE

Lab 1: Run Backstage locally

Use npx or Docker to launch Backstage.

Lab 2: Register a service

Add a catalog-info.yaml file.

Lab 3: Create a template using Scaffolder

Template includes:

- CI pipeline
- Dockerfile
- K8s deployment

Lab 4: Create scorecards

Examples:

- SBOM present?
- CI pipeline exists?
- Image signed?
- Terraform validated?

WHERE TO STUDY

- Backstage documentation
- Scaffold template examples
- Scorecards plugin

INFRATHrone MINDSET NOTES

- Platform $\approx$ internal product $\rightarrow$ measure usage, adoption, friction.
- Scorecards guide behavior better than strict blocking.
- Self-service reduces wait times $\rightarrow$ increases dev happiness.

